

idea.t...

backe...

We have finalized the concept of a Capability Assurance Operating System for banks.

Help me design the backend architecture.

Requirements:

- Multi-agent workflow
- Human approval checkpoints
- Auditability
- Regulatory traceability
- Long-running workflows
- Evidence lineage

Please propose:

1. Core backend services



CodeBuddy

Deep Thinking

Read idea.txt L1-End

Deep Thinking

Creating a formal backend architecture document for the Capability Assurance Operating System, mapping the concept from idea.txt into concrete services, responsibilities, data flows, and AI/deterministic boundaries.

ARCHITECTURE.md (Write) +322 -0

View File

Deep Thinking

I've created a formal backend architecture document at:

/Users/chijian/Desktop/TUNTAS/backend/ARCHITECTURE.md

Here is a summary of the design:

1. Core Backend Services (6 Layers)

Service	Role
Gateway & Identity	SSO, RBAC, routing, tenant isolation
Workflow Orchestration	Durable state machine, approval routing, human checkpoints, escalation

Task List 6/6

File List (0)

All tasks completed

Review the complete frontend experience.

Please help me optimize it for a hackathon demo.

I want the product to communicate:

- AI innovation
- trustworthiness
- governance
- regulatory readiness

Provide:

1. Recommended demo sequence
2. Most impressive screens
3. Potential UX weaknesses
4. Quick improvements before judging

Assume the audience consists of banking executives, compliance leaders, and technical judges.



Deep Thinking

Review and optimize the complete frontend experience for a hackathon demo, covering demo sequence, impressive screens, UX weaknesses, and quick improvements

HACKATHON_DEMO_OPTIMIZATION.md (Write) +457 -0

View File

Deep Thinking

- Read HACKATHON_DEMO_OPTIMIZATION.md L1-5
- Read HACKATHON_DEMO_OPTIMIZATION.md L300-399

Deep Thinking

- Read HACKATHON_DEMO_OPTIMIZATION.md L395-494

Deep Thinking >

Review saved to frontend/HACKATHON_DEMO_OPTIMIZATION.md . Highlights:

The optimized demo sequence (4:30, timed to the second) A single narrative with one protagonist (Alice Chen, Compliance Officer) and one antagonist (the regulator's audit). The story: *"The bank must prove to a regulator that 250 employees are ready for a new BNM regulation. The OS does this with AI, human oversight, and cryptographic proof."*

Minute	Beat	Screen	Message
0:00	Hook	Presenter	Problem

Design a Governance and Approval Workspace.

Requirements:

- compare portfolio options
- review objections
- attach conditions
- approve
- reject
- request changes

Every action must be auditable.

Describe:

- layout
- workflow
- decision support features
- compliance considerations



CodeBuddy

Deep Thinking

Design the Governance and Approval Workspace with layout, workflow, decision support, and compliance considerations grounded in the backend approval infrastructure

APPROVALS_WORKSPACE.md (Write) +570 -0

View File

Deep Thinking

Read APPROVALS_WORKSPACE.md L1-5

Read APPROVALS_WORKSPACE.md L300-399

Deep Thinking

Read APPROVALS_WORKSPACE.md L450-549

Deep Thinking

Read APPROVALS_WORKSPACE.md L549-598

Deep Thinking

Design saved to frontend/APPROVALS_WORKSPACE.md . Highlights:

The core design decision: a control surface, not a form The user must feel they are operating a critical system where the system watches them as closely as they watch it. This is the trust loop: the user trusts the system because the system does not trust the user.

Layout: four zones, one decision surface



Design a visual experience for exploring evidence lineage.

A user should be able to start from:

"Employee readiness"

and trace backwards to:

- simulations
- training
- competencies
- controls
- policy clauses

Describe:

- interaction model
- graph visualization options
- drill-down behavior



Deep Thinking

Design a visual experience for exploring evidence lineage, including interaction model, graph visualization, drill-down behavior, and audit workflow.

EVIDENCE_LINEAGE.md (Write) +182 -0

View File

Deep Thinking

Read EVIDENCE_LINEAGE.md L1-50

Deep Thinking

Read EVIDENCE_LINEAGE.md L50-149

Deep Thinking

Read EVIDENCE_LINEAGE.md L1-5

Deep Thinking

Read EVIDENCE_LINEAGE.md L1-End

Deep Thinking >

Delete truncated file to rewrite properly

frontend/EVIDENCE_LINEAGE.md Deleted

Deep Thinking

.....

I want users to observe how the AI workflow progresses.

Design an Agent Timeline experience.

Requirements:

- show agent execution
- show handoffs
- show evidence creation
- show approval checkpoints

Avoid fake animations.

The timeline should reflect actual backend events and make the system feel trustworthy.



CodeBuddy

Deep Thinking

Listed backend/src/capos/agents >

Read state.py L1-End

Read interrupt.py L1-End

Deep Thinking

Read contracts.py L1-End

Read constants.py L1-End

Read persistence.py L1-End

Read graph.py L1-End

Deep Thinking

Read workflow.py L1-End

Listed backend/src/capos/domains/audit >

Deep Thinking

I have full grounding in the backend event sources (state_history , artifacts , evidence_bundles , human_decisions , pending_checkpoint , hash-chained transition_hash). Designing the Agent Timeline around real backend events, not decoration.

AGENT_TIMELINE.md (Write) +581 -0

View File

Deep Thinking

Agent Timeline design saved to frontend/AGENT_TIMELINE.md . Key points:



Task List 6/6

File List (0)

All tasks completed

Design the main Management Decision Cockpit.

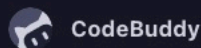
This is the screen executives will spend most of their time in.

Requirements:

- portfolio comparison
- budget utilization
- risk reduction visibility
- regulatory coverage
- approval actions

Describe:

- layout
- information hierarchy
- widgets
- visual emphasis



Deep Thinking

I'll design this as a focused screen specification — one main screen, every widget justified, every pixel earning its place.

COCKPIT_DESIGN.md (Write) +614 -0

View File

Deep Thinking

Cockpit design saved to `frontend/COCKPIT_DESIGN.md`. Highlights:

The metaphor — cockpit, not dashboard A dashboard reports; a cockpit supports a decision that's about to happen. The cockpit must answer three questions in 5 seconds: are we on track, where do I focus, what does the OS need from me.

The four zones (F-pattern + persistent right rail)

code

Copy Insert New File Save Apply

```
ZONE A: TICKER (top, 96px, always on) – 5 gauges + regulatory pulse
ZONE B: PORTFOLIO COMPARISON (center, main canvas) – 12-row table OR heatmap
ZONE C: CONTEXT PANEL (bottom, 220px) – risk trend | budget burn | regulatory map
ZONE D: ACTION QUEUE (right rail, 360px, sticky, never collapses)
```

The right rail is **sacred** — the action queue is the screen's reason for existing. Executives came here to act, not to be informed.



Task List 6/6

File List (0)

Help me create a complete screen inventory for the MVP.

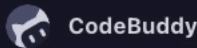
Requirements:

- enterprise-grade UX
- demo-friendly
- clear storytelling

For each screen provide:

- purpose
- primary actions
- key data displayed
- user role

Focus on the minimum set of screens needed for a compelling hackathon demo.



Deep Thinking

I'll design the minimum set of screens that tell a complete story in ~5 minutes. The key is ruthless prioritization — every screen must earn its place in the demo.

MVP_SCREEN_INVENTORY.md (Write) +680 -0

View File

Deep Thinking

Screen inventory saved to `frontend/MVP_SCREEN_INVENTORY.md`. Highlights:

18 screens total — not 50

The IA had ~50 screens. For the MVP, I cut to **18** by ruthless prioritization: every screen must serve the demo narrative. The 50 → 18 cut is the most important decision.

Tier	Count	Definition
Hero	6	The "wow" screens; presenter slows down here
Flow	8	The work screens; clicked through quickly
Context	4	Anchor screens; brief but present

The 5-minute narrative arc (the demo script)

The 18 screens sequence into a 5-minute story with rising tension that peaks at the **Lineage Graph** (screen 13) and resolves with the **Regulator Bundle** (screen 14):

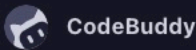
Based on the Information Architecture, design the major user journeys.

Key scenarios:

- 1. Launch a capability planning run
- 2. Compare portfolio options
- 3. Review evidence lineage
- 4. Approve or reject recommendations
- 5. Investigate readiness claims
- 6. Respond to policy changes

For each journey provide:

- user steps
- decisions
- required screens
- success criteria



Deep Thinking

I'll design the 6 major user journeys as a complete, scannable doc that maps each journey to specific state transitions and screen-level affordances.

✚ USER_JOURNEYS.md (Write) +730 -0

View File

Deep Thinking

Journeys saved to `frontend/USER_JOURNEYS.md`. Highlights:

Six journeys, one binding principle Every journey is a sequence of **state mutations**, not a sequence of pages. The screens serve the journey; the audit trail is the journey.

#	Journey	Primary actor	Default workspace	Time budget
1	Launch a planning run	HR / L&D	Plan	20–40 min first, 5 min repeat
2	Compare portfolio options	CRO + Risk	Plan → Risk	10–30 min
3	Review evidence lineage	Compliance	Evidence Library	2–10 min, or full audit day
4	Approve / reject	role-dependent per checkpoint	Approvals	2–10 min per checkpoint
5	Investigate readiness claims	Compliance	Evidence Library	2–10 min, or full audit day
6	Respond to policy changes	Compliance	Evidence Library	2–10 min, or full audit day

Task List 6/6

File List (0)

All tasks completed

The backend architecture is now defined.

Before designing screens, help me design the Information Architecture (IA) for a Capability Assurance Operating System.

Users include:

- Management
- HR
- Compliance
- Risk teams

Please propose:

1. Primary navigation structure
2. Core workspaces
3. Key user goals
4. How information should flow through the product



CodeBuddy

Deep Thinking

Let me peek at the frontend folder to ground the IA in any existing structure.

Listed frontend >

Deep Thinking

The frontend is empty so I'm designing greenfield. Here's the IA design.

📄 INFORMATION_ARCHITECTURE.md (Write) +609 -0

View File ↕

Deep Thinking

IA design saved to `frontend/INFORMATION_ARCHITECTURE.md`. Highlights:

Core design decision — "one OS, role overlays" Roles do not get their own workspaces. They get **affordances** within the same seven workspaces. This is what makes it an OS rather than four portals bolted together. A workflow in the `Plan` workspace is the same object the HR person launched, the Compliance officer is reviewing, and the Risk person is monitoring — they're all looking at the same state with role-scoped permissions.

The shell — four regions, every screen

- **Top Bar** — workspace switcher, ⌘K command bar, notifications, identity
- **Left Rail** — object tree (workflows / employees / controls / evidence), not page navigation
- **Object Canvas** — the active object with sub-tabs (Overview / Lineage / Decisions / Audit)
- **Right Inspector** — always-on context (collaborators, activity, lineage, open questions)



Task List 6/6 ▾

File List (0) ▾

All tasks completed

Help me implement the portfolio optimization engine.

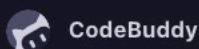
Requirements:

- RM5,000 employee cap
- RM1.25m total cap
- Q3 scheduling
- prerequisite constraints
- course capacities
- minimum business coverage

Explain:

- OR-Tools model structure
- variables
- constraints
- objective functions

I want implementation guidance rather than theory.



Deep Thinking

Let me explore the optimization domain and the agent outputs that feed into it.

Listed backend/src/capos/domains/optimization >

Listed backend/src/capos/domains/workforce >

Listed backend/src/capos/domains/intervention >

🔍 Read contracts.py L1-End

Deep Thinking

Let me look at the optimization node in the graph and the existing optimize endpoint to see the expected input shape.

🔍 Search failed optimization_result|optimization_node|run_optimization|OptimizationEngine|ortools >

🔍 Searched def optimization_node|optimization_result\s*=\|OR-Tools|ortools >

🔍 Read nodes.py L115-119

🔍 Read state.py L1-End

🔍 Read optimize.py

🔍 Read models.py

🔍 Read service.py



Task List 6/6 ▾

File List (0) ▾

All tasks completed

Design the implementation of a mandatory human approval workflow.

Requirements:

- approve
- reject
- request changes
- select alternative

I need:

- API design
- database schema
- workflow integration
- audit event generation

The workflow must prevent downstream execution before approval.



CodeBuddy

Deep Thinking

I'll explore the codebase first to design this approval workflow that integrates properly with the existing evidence spine, agent contracts, and LangGraph workflow.

Task code-explorer subagent: Explore approval/checkpoint/audit patterns

- Show `backend/src/capos/api/router.py` or main route registration

6. Settings & configuration:

- Show `backend/src/capos/core/settings.py` or `config.py`
- Find any role definitions (e.g., `COMPLIANCE_REVIEWER`, `APPROVER`)

7. Existing domains layout:

- List the contents of `backend/src/capos/domains/` directory
- Show the structure of an existing domain folder (e.g., `evidence/`) — `models.py`, `repository.py`, `service.py`, `schemas.py` if present

For each finding, return: file path, line range, and a brief snippet. Be thorough — I need a complete picture to design an approval workflow that integrates cleanly. Do NOT make changes; this is a read-only exploration.

Execution Summary: 34 tool uses, cost: 74.92s, credits: 5.09

Deep Thinking

Let me read the key files directly to get exact details for the design.



Task List 6/6

File List (0)

All tasks completed

I want to implement the Evidence Spine.

Please design:

- node model
- edge model
- lineage queries
- evidence retrieval APIs

The system should support tracing:

- Employee Readiness
 - Simulation Result
 - Training Activity
 - Competency
 - Control
 - Policy Clause

Focus on implementation details rather than conceptual explanations.



CodeBuddy

Deep Thinking

🕒 Read graph.py L1-End

🔍 Searched files backend/src/capos evidence >

Deep Thinking

🕒 Read database.py L1-End

🕒 Read workflow.py L1-End

🕒 Read router.py L1-End

Deep Thinking

Listed backend/src/capos/domains/evidence >

Listed backend/src/capos/api/v1 >

🕒 Read constants.py L1-End

Deep Thinking

☰ Task List 6/6 done

- ✓ Add EvidenceEntityType enum and update constants.py
- ✓ Create Pydantic models for evidence nodes and edges
- ✓ Create EvidenceGraphRepository with lineage queries (recursive CTE)



Task List 6/6 ▾

File List (0) ▾

All tasks completed

Help me define implementation-ready contracts between agents.

Requirements:

- strict schema validation
- typed outputs
- traceable evidence references
- confidence tracking

For each agent output, propose a JSON schema structure that can be validated before handoff.



CodeBuddy

Deep Thinking

👁 Read contracts.py L1-End

Deep Thinking

👁 Read constants.py L1-End

Deep Thinking

👁 Read nodes.py L1-End

Deep Thinking

👁 Read state.py L1-40

Deep Thinking

☰ Task List 4/4 done

- ✓ Rewrite contracts.py with provenance, evidence refs, confidence tracking, and strict validation
- ✓ Create agents/validation.py with pre-handoff ContractValidator
- ✓ Update nodes.py to populate new provenance/confidence fields using helper functions
- ✓ Create docs/AGENT_CONTRACTS.md with JSON schemas and validation rules

Deep Thinking

Rewriting contracts.py with BaseAgentOutput enforcing provenance, evidence_refs, confidence, and output_hash.
Adding strict validators, factory helpers, and per-item confidence fields.

🔗 contracts.py (Write) +651 -110

View File ↕

Deep Thinking

☰ Task List 4/4 done



Task List 6/6 ▾

File List (0) ▾

All tasks completed

I want to implement the workflow engine first.

Using LangGraph concepts, design:

- graph state
- node responsibilities
- agent handoff contracts
- persistence strategy
- interrupt/resume flow

Focus on implementation details that can be directly translated into code.



CodeBuddy

Deep Thinking

Listed backend/src/capos >

Deep Thinking

🕒 Read state.py L1-End

🕒 Read graph.py L1-End

🕒 Read engine.py L1-End

🕒 Read dependencies.py L1-End

🕒 Read constants.py L1-End

Deep Thinking

Pydantic v2 schemas defining strict agent handoff contracts between all nodes in the workflow graph.

🔗 contracts.py (Write) +291 -0

View File ▾

Deep Thinking

🔗 state.py (Write) +104 -21

View File ▾

Deep Thinking

🔗 nodes.py (Write) +653 -0

View File ▾

Deep Thinking

Complete LangGraph builder with conditional entry point (resume support), all 17 workflow states mapped to nodes, and conditional routing after every human checkpoint.

🔗 graph.py (Write) +270 -177

View File ▾



Deep Thinking

Let's start implementation.

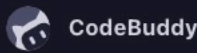
Design a production-quality repository structure for the backend.

Technology stack:

- Python 3.12
- FastAPI
- LangGraph
- Pydantic v2
- Supabase
- OR-Tools

Please propose:

- folder structure
- module boundaries
- separation of concerns
- dependency injection strategy



Deep Thinking

Listed backend >

Deep Thinking

Creating the complete directory structure for the production backend repository.

```
mkdir -p /Users/chijian/Desktop/TUNTAS/backend/src/capos/{core,api/v1,api/middleware,domains/{identi
ty,regulation,workforce,gap,intervention,workflow,optimization,evidence,audit},agents/prompts,infrastr...
```

Deep Thinking

Modern pyproject.toml with Python 3.12, FastAPI, LangGraph, Pydantic v2, Supabase, OR-Tools, and dev tooling.

```
pyproject.toml (Write) +101 -0 View File
```

Deep Thinking

```
.env.example (Write) +24 -0 View File
```

Deep Thinking

Docker Compose for local PostgreSQL and Redis fallback during development.

```
! docker-compose.yml (Write) +33 -0 View File
```

Deep Thinking

```
config.py (Write) +58 -0 View File
```

I want to build a hackathon MVP within a limited timeline.

Please break the backend into implementation phases.

For each phase provide:

- scope
- deliverables
- dependencies
- success criteria

Assume I am using:

- FastAPI
- LangGraph
- PostgreSQL/Supabase
- Gemini
- OR-Tools



CodeBuddy

Deep Thinking

Creating a hackathon implementation roadmap that breaks the backend into 4 phases with scope, deliverables, dependencies, and success criteria, tailored to FastAPI + LangGraph + Supabase + Gemini + OR-Tools.

IMPLEMENTATION_ROADMAP.md (Write) +299 -0

View File

Deep Thinking

I've created the implementation roadmap at:

/Users/chijian/Desktop/TUNTAS/backend/IMPLEMENTATION_ROADMAP.md

Here is the phase breakdown:

Phase 1: Foundation & Intake (Day 1)

Scope: Scaffold, connect Supabase, core schema, seeded data, deterministic gap analysis.

Deliverables:

- FastAPI project + Pydantic models + Supabase client
- Core SQL schema: regulation, policy_clause, control, employee, proficiency_record, capability_gap, workflow_instance
- Seeded data: 250 employees across 4 roles, BNM regulations, controls, proficiency levels
- POST /api/v1/workflow/intake → spawns workflow
- POST /api/v1/analysis/gap → computes gap matrix (current vs. required level)



Task List 6/6

File List (0)

All tasks completed

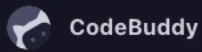
Based on the architecture above, help me design:

1. Core database entities
2. Workflow lifecycle
3. State transitions
4. Evidence relationships

I want a backend that can explain:

Why a training recommendation was made,
Who approved it,
What evidence supports it.

Please focus on implementation-ready data structures.



Deep Thinking

Creating an implementation-ready data model document with SQL schemas, workflow state machine, state transitions, approval checkpoints, evidence graph, and immutable event ledger for the Capability Assurance Operating System.

DATA_MODEL.md (Write) +997 -0

View File

Deep Thinking

I've created a comprehensive implementation data model at:

/Users/chijian/Desktop/TUNTAS/backend/DATA_MODEL.md

Here is a structured summary of the design:

1. Core Database Entities (20+ Tables)

Identity & Governance

- `tenant` — multi-tenant boundary (banks, subsidiaries)
- `system_user` — employees, agents, system integrations with workflow roles
- `workflow_permission` — RBAC guard matrix

Regulation & Policy

- `regulation` — versioned regulation records (BNM RMiT, AML/CFT, PDPA)
- `policy_clause` — internal policy broken into clauses, hierarchical



Task List 6/6

File List (0)

All tasks completed